Machine Learning Pipeline for 15-Minute Ahead Solar Power Generation Forecasting

Course	GenAI Capstone Project
Dataset	Kaggle Solar Power Generation Data (Plant 1 & 2)
Repository	github.com/Aviral02git/Solar-power-prediction
Deployed App	solarpowerpredictionmodel.streamlit.app
Date	April 2026

Member	Role	Contributions
Ved	Tech Lead	Model development, feature engineering, Streamlit deployment, system integration
Aviral Mishra	Data & Quality Lead	Dataset preparation, data quality checks, preprocessing pipeline
Naitik Pandey	Report & Evaluation Lead	Model evaluation, metric analysis, project documentation
Samarth Khera	Analysis Lead	Trend analysis, visualisation, insights from results

Contents

1. Introduction & Problem Statement

Solar energy generation is among the most cost-effective and scalable forms of renewable power, yet its output is governed by meteorological conditions that change continuously. Cloud cover, irradiance fluctuations, temperature gradients, and diurnal cycles mean that plant-level DC power can vary by orders of magnitude within minutes. Grid operators cannot absorb this variability passively — they require short-horizon forecasts to dispatch backup generation, balance storage charge cycles, and preserve frequency stability.

This project builds a complete machine learning pipeline that ingests raw inverter-level generation records and weather sensor readings from a real solar plant and produces a 15-minute ahead DC power forecast. The system is deployed as an interactive web application through which a grid operator can upload daily plant data, inspect model performance, and obtain structured optimisation recommendations from an autonomous agentic assistant.

Core problems addressed:

- Strong time-dependency in power output — successive 15-minute intervals are highly correlated, requiring lag-based features to capture temporal structure.
- Nonlinear relationships between irradiance, ambient temperature, module temperature, and plant-level generation that linear models cannot capture reliably.
- High variance during sunrise and sunset transition periods, where small changes in solar angle produce large swings in output.
- Noise in inverter-level measurements that must be aggregated before modelling.
- Grid operators need actionable, short-term guidance — not just a number — to allocate storage and schedule demand response.

2. System Evolution: Milestone 1 → Milestone 2

The project was structured across two milestones. Milestone 1 delivered a classical ML forecasting pipeline; Milestone 2 extended that pipeline with an agentic AI layer capable of reasoning over forecast outputs and producing structured grid management recommendations.

Table 1: Component-level comparison across milestones

Component	Milestone 1	Milestone 2
Core task	15-min solar power forecasting	Agentic grid optimisation assistant
Model	RandomForestRegressor	RandomForest + LangGraph + LLM
Retrieval	None — static trained model	FAISS vector search + MiniLM embeddings
Pipeline	Batch inference, single step	Multi-node agentic workflow
Output	Numerical forecast + evaluation metrics	Structured optimisation recommendations
Reasoning	None (statistical inference)	Multi-step LLM reasoning via Groq
UI	Streamlit — Tab 1 only	Streamlit — Tab 1 (forecast) + Tab 2 (agent)

3. Dataset

The dataset was sourced from Kaggle's Solar Power Generation Data collection, which contains real operational records from two solar plants in India, sampled at 15-minute intervals over a 34-day period in May–June 2020.

Table 2: Dataset files and key fields

File	Key Fields	Description
`Plant_1_Generation_Data.csv`	DATE_TIME, DC_POWER, PLANT_ID, SOURCE_KEY	Inverter-level DC power at 15-min intervals
`Plant_1_Weather_Sensor_Data.csv`	DATE_TIME, AMBIENT_TEMPERATURE, MODULE_TEMPERATURE, IRRADIATION	Environmental sensor readings at plant level
`Plant_2_Generation_Data.csv`	Same schema as Plant 1	Second plant used for cross-plant validation
`Plant_2_Weather_Sensor_Data.csv`	Same schema as Plant 1	Weather data for Plant 2

Plant 1 contains 68,778 inverter-level records across 22 inverters, which aggregate to 3,157 plant-level 15-minute timestamps after groupby summation. The weather sensor provides one row per 15-minute interval at the plant level.

4. Methodology

4.1 Data Aggregation and Integration

Inverter-level `DC_POWER` readings are summed by timestamp to produce plant-level output — a necessary step because the model must forecast total plant generation, not per-inverter output. The aggregated generation table is then merged with the weather sensor table on `DATE_TIME`, and the result is sorted chronologically and stripped of any rows containing nulls.

4.2 Feature Engineering

Nine features were constructed from the merged dataset. Three capture time periodicity (`hour`, `day_of_year`, `month`); three capture temporal autocorrelation in power output (`lag_1`, `lag_4`, `rolling_mean_4`); and three are direct environmental readings (`AMBIENT_TEMPERATURE`, `MODULE_TEMPERATURE`, `IRRADIATION`).

The target is formed by shifting `DC_POWER` forward by one row, so each sample represents the current state of the plant and the label is the plant's output in the next 15-minute interval.

4.3 Model Training

A `RandomForestRegressor` was selected as the primary model. Random forests handle nonlinear feature interactions natively through recursive partitioning, require minimal preprocessing, and are robust to the noisy inverter-level measurements present in the raw data. The trained ensemble was persisted to `solar_forecast_model.pkl` via `joblib` for inference in the Streamlit application.

Hyperparameters:

- `n_estimators = 200`
- `max_depth = 12`

Table 3: Feature engineering summary

Feature Type	Features	Rationale
Time-based	`hour`, `day_of_year`, `month`	Captures diurnal and seasonal periodicity
Lag features	`dc_power_lag1` $(t-1)$, `dc_power_lag4` $(t-4)$	Recent history is the strongest predictor of near-future output
Rolling statistic	`rolling_mean_4` (4-interval window)	Smooths short-term noise; tracks trend momentum
Weather features	`AMBIENT_TEMPERATURE`, `MODULE_TEMPERATURE`, `IRRADIATION`	Physical drivers of photovoltaic conversion efficiency
Target variable	`DC_POWER` shifted -1 interval	Defines the 15-minute ahead prediction objective

- `random_state = 42`
- `n_jobs = -1` (parallelised across all cores)

Training was performed on the first 80% of chronologically ordered records (2,521 samples), with the remaining 20% (631 samples) held out for evaluation. A time-ordered split — rather than random shuffling — was used to prevent data leakage across the temporal boundary.

5. System Architecture

5.1 Milestone 1 — Batch Inference Pipeline

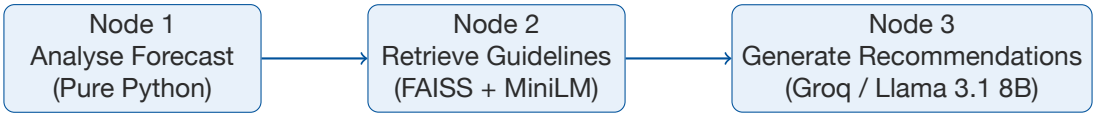
The Milestone 1 system follows a five-layer architecture in which each layer has a single, well-defined responsibility.

Table 4: Architectural layers

#	Layer	Responsibility
1	Data Layer	Ingestion of raw Plant Generation and Weather Sensor CSVs via Streamlit file uploader
2	Feature Engineering Layer	Aggregation, timestamp alignment, lag features, rolling statistics, time-based features
3	Machine Learning Layer	`RandomForestRegressor` inference using `solar_forecast_model.pkl`
4	Evaluation Layer	MAE, RMSE, and R^2 computed against the held-out target column
5	Application Layer	Streamlit UI rendering forecast chart, trend analysis, and downloadable predictions CSV

5.2 Milestone 2 — Agentic Workflow Layer

Milestone 2 introduces a **LangGraph** agentic layer that operates downstream of the forecasting pipeline. After the Milestone 1 pipeline produces predictions, the agent receives those predictions as input and executes a three-node directed workflow.



Node 1 — Analyse Forecast (pure Python, no external calls): Receives the full prediction array and irradiation series. Computes mean, max, min, standard deviation, and variability percentage. Classifies grid risk as Low ($< 20\%$ variability), Medium (20–40%), or High ($> 40\%$). Identifies risk periods where output drops more than 30% below the mean, and peak hours in the top decile of predicted output.

Node 2 — Retrieve Guidelines (FAISS + sentence-transformers): Constructs a semantic query from the Node 1 analysis output and performs a top-4 similarity search over an in-memory FAISS index built from 18 grid management guideline chunks. Embeddings are generated locally using `sentence-transformers/all-MiniLM-L6-v2`, with no external embedding API dependency.

Node 3 — Generate Recommendations (Groq API, Llama 3.1 8B): Sends the structured analysis and the four retrieved guideline passages to Llama 3.1 8B via the Groq inference API. The model is prompted to return structured output — a forecast summary, risk assessment, immediate actions, optimisation strategies, and referenced guideline topics — without markdown formatting. JSON fence stripping is applied to the response before parsing.

6. Technology Stack

Table 5: Full technology stack across both milestones

Category	Milestone 1	Milestone 2 (additional)
Language	Python 3.11	Python 3.11
ML / Modelling	scikit-learn (RandomForestRegressor)	LangGraph, LangChain
Vector search	—	FAISS + sentence-transformers (all-MiniLM-L6-v2)
LLM inference	—	Groq API (Llama 3.1 8B)
Data processing	Pandas 3.0, NumPy 2.4	Pandas 3.0, NumPy 2.4
Visualisation	Matplotlib 3.10	Matplotlib 3.10
Web UI	Streamlit 1.54 — single tab	Streamlit 1.54 — two tabs
Model storage	Joblib (.pkl)	Joblib (.pkl)
Secrets	`python-dotenv`	`python-dotenv` + Streamlit secrets
Dev environment	Google Colab	Google Colab / local venv

7. Evaluation & Results

Model performance was evaluated on the chronological 20% hold-out split using three standard regression metrics. Additionally, a daytime-only evaluation was performed by filtering to hours with non-zero irradiation, isolating the model's performance under the conditions where forecasting accuracy is most operationally relevant.

An R^2 of 0.9905 on daytime-only data confirms that the model captures the underlying photovoltaic generation process with high fidelity when irradiance is stable. The drop to 0.9323

Table 6: Model evaluation results

Evaluation Split	MAE	RMSE	R^2	Observation
Daytime only	4,646.83	7,397.92	0.9905	Near-perfect accuracy under stable irradiance conditions
Full dataset	10,573.81	21,207.71	0.9323	Reduced accuracy driven by sunrise and sunset transitions

on the full dataset reflects the difficulty of transition periods — dawn and dusk introduce steep, short-lived power ramps that are structurally harder to predict with lag-based features trained primarily on steady-state conditions.

The most informative feature, by a wide margin, was `IRRADIATION` (mean decrease in impurity > 0.8). Lag features and rolling mean contributed the next largest shares, confirming that temporal autocorrelation in power output carries predictive signal independent of the weather variables.

8. Streamlit Application

The application is structured across two tabs, each handling a distinct stage of the analysis workflow.

Table 7: Application features by tab

Tab	Feature	Description
Tab 1 — Forecast	CSV upload	Accepts Plant Generation and Weather Sensor CSVs; validates required columns
Tab 1 — Forecast	Auto preprocessing	Aggregation, merge, feature engineering, and model inference applied automatically
Tab 1 — Forecast	15-min forecast chart	Actual vs. predicted DC power over a user-selected window (50–1,000 points)
Tab 1 — Forecast	Performance metrics	MAE, RMSE, and R^2 displayed as metric cards
Tab 1 — Forecast	Anomaly detection	Points where prediction error exceeds a configurable threshold are flagged and highlighted
Tab 1 — Forecast	Trend analysis	Hourly bar chart and irradiation scatter plot
Tab 1 — Forecast	Download predictions	Exports timestamp, actual, predicted, error, and anomaly flag as CSV
Tab 2 — AI Agent	Run Analysis	Triggers the LangGraph pipeline; results persist in session state across reruns
Tab 2 — AI Agent	Grid recommendations	Structured risk assessment, actions, and optimisation strategies from Llama 3.1 8B
Tab 2 — AI Agent	Follow-up chat	Conversational interface for querying the analysis using RAG context
Tab 2 — AI Agent	PDF report	Exports a LaTeX-compiled PDF containing analysis statistics and recommendations

Run locally:

```
python3 -m streamlit run app.py
```

9. Design Decisions & Tradeoffs

Every architectural choice in this project involved a tradeoff between capability, complexity, and deployment constraints. The decisions below reflect practical engineering priorities for a capstone project that must be reproducible, demonstrable, and academically rigorous.

Table 8: Key design decisions with rationale and tradeoffs

Decision	Rationale	Tradeoff
RandomForest over LSTM	Handles nonlinear tabular data; straightforward to train, interpret, and deploy without GPU	No explicit temporal memory; LSTM or TCN would model long-range dependencies better
Batch inference over real-time	Lower infrastructure complexity; appropriate for academic demonstration on static CSVs	Cannot respond to streaming grid inputs; not suitable for production without an API layer
Groq / Llama 3.1 8B for Milestone 2	Fast inference, free-tier availability, strong instruction-following on structured output tasks	External API dependency; potential latency under load; cost at production scale
FAISS for RAG retrieval	Local, in-memory vector search with no external database dependency; zero network calls	Static knowledge base — index must be rebuilt to incorporate new guidelines
Streamlit for UI	Rapid prototyping; one-command deployment to Streamlit Cloud; Python-native	Batch-only interaction model; no WebSocket support for streaming inference

10. Limitations & Future Work

11. Repository Structure

```
Genai capstone/
  app.py           Unified Streamlit app (Tab 1: forecast, Tab 2: AI agent)
  agent.py         LangGraph 3-node agentic workflow + chat function
  rag.py           FAISS vector store and retrieval function
  knowledge_base.py 18 grid management guideline text chunks
  report.tex       This document
  requirements.txt   Pinned Python dependencies
  data/
    Plant_1_Generation_Data.csv
    Plant_1_Weather_Sensor_Data.csv
    Plant_2_Generation_Data.csv
```

Table 9: Current limitations and planned improvements

Category	Current Limitation	Planned Improvement
Forecasting	Batch-only; no real-time data ingestion	Real-time streaming via Kafka + REST API layer
Model	No hyperparameter tuning	Add GridSearchCV / Optuna HPO pipeline
Model	No cross-validation pipeline	Implement time-series cross-validation (TimeSeriesCV)
Architecture	No model drift detection	Add monitoring dashboard for production drift
Architecture	No REST API endpoint	Deploy model as FastAPI REST service
ML Approach	RandomForest lacks temporal memory	Implement LSTM or Temporal Convolutional Networks
Data	No live weather forecast integration	Integrate weather API for real-time future inputs
LLM (M2)	API latency and cost at scale	Cache recommendations; explore local LLM hosting
LLM (M2)	Potential hallucination in recommendations	Add citation validation and fact-checking layer

```
    Plant_2_Weather_Sensor_Data.csv
  milestone_1/
    app.py        Original Milestone 1 Streamlit app
    solar_model.ipynb Training notebook (RandomForest)
    ARCHITECTURE.md
```

12. Conclusion

Milestone 1 delivers a complete, deployable machine learning pipeline for short-term solar power forecasting. A RandomForestRegressor trained on temporally engineered features from real plant operational data achieves an R^2 of 0.9905 under stable daytime conditions and 0.9323 across the full dataset — performance sufficient for operational use in grid scheduling applications.

Milestone 2 extends this system into an agentic platform. Rather than producing a number and stopping, the system now analyses forecast variability, retrieves relevant grid management guidelines from a curated knowledge base, and synthesises structured recommendations through multi-step LLM reasoning. This positions the system as a decision support tool for grid operators, not merely a forecasting service.

The architecture is modular at every layer: the ML pipeline, the vector store, the LLM integration, and the UI are independently replaceable components. Substituting the RandomForest with an LSTM, swapping FAISS for a hosted vector database, or replacing Groq with a locally hosted model each require changes to a single module. This modularity is the principal engineering contribution of the project — it demonstrates how a GenAI system can be constructed from composable, well-bounded components rather than as a monolithic pipeline.